

I. Forecasting of Time Series — Scientific Expansion

Time series forecasting is a central topic in modern data science, econometrics, and quantitative finance. A **time series** is a sequence of observations indexed by time, typically denoted by

$$\{x_t\}_{t=1}^N,$$

where t is a discrete time index (e.g., days, weeks, months) and $x_t \in \mathbb{R}$ represents the observed value at time t . In financial applications, x_t often corresponds to a stock price, an index level, a log-return, or a volatility measure.

The core forecasting problem can be formulated as follows:

Given a set of well-prepared data points $\{x_t\}_{t=1}^N$ with respect to a discretized real-valued time parameter t , what should be the values $x_{t=N+m}$ for

$$m = 1, \dots, K, \quad K < \infty, \quad K \geq 1,$$

where K is the **length of the future horizon**, i.e., the number of time units (typically days or weeks) that one aims to forecast?

This question lies at the heart of time series analysis: how to extrapolate from past observations into the future in a way that is **statistically sound**, **computationally feasible**, and **robust** to noise and structural changes.

1. Discrete Time Series and Stock Price Evolution

In the context of stock markets, the evolution of a stock price x_t over time is a prototypical example of a discrete time series. Let us consider a sequence of closing prices:

$$x_1, x_2, \dots, x_N,$$

where each x_t is the closing price of a given stock at day t . The goal is to forecast:

$$x_{N+1}, x_{N+2}, \dots, x_{N+K},$$

based on the historical data. In practice, one often works not directly with prices but with **log-returns**:

$$r_t = \log \left(\frac{x_t}{x_{t-1}} \right),$$

because returns tend to be more stationary than raw prices, which is a crucial requirement for many classical time series models.

From a data-analytic perspective, the first step is to **extract trends, seasonality, and residual components** from the observed series. A common decomposition is:

$$x_t = T_t + S_t + R_t,$$

where:

- T_t is the **trend component** (long-term movement),
- S_t is the **seasonal component** (periodic fluctuations),

- R_t is the **residual or noise component**.

The forecasting task then becomes: model T_t , S_t , and R_t in such a way that future values x_{N+m} can be predicted with minimal error.

2. Formal Statement of the Forecasting Problem

Let us denote the observed time series as:

$$\mathcal{D}_N = \{(t, x_t) \mid t = 1, \dots, N\}.$$

We seek a forecasting function F such that:

$$\hat{x}_{N+m} = F(\mathcal{D}_N, m), \quad m = 1, \dots, K,$$

where $\hat{x}_{N+m}$ is the forecast for time $N + m$. The function F may be:

- a **parametric statistical model** (e.g., ARIMA, SARIMA),
- a **machine learning model** (e.g., random forest, gradient boosting, neural networks),
- a **hybrid model** combining statistical and ML components.

The quality of the forecasting function is typically evaluated via loss functions such as:

$$\text{MSE} = \frac{1}{K} \sum_{m=1}^K (x_{N+m} - \hat{x}_{N+m})^2,$$

$$\text{MAE} = \frac{1}{K} \sum_{m=1}^K |x_{N+m} - \hat{x}_{N+m}|,$$

or more sophisticated metrics tailored to financial applications (e.g., directional accuracy, hit rate, risk-adjusted measures).

3. SARIMA and Classical Time Series Models

A widely used class of models for time series forecasting is the **ARIMA** family (AutoRegressive Integrated Moving Average). In its seasonal extension, **SARIMA** (Seasonal ARIMA), the model can capture both non-seasonal and seasonal dynamics.

A general SARIMA model is denoted as:

$$\text{SARIMA}(p, d, q) \times (P, D, Q)_s,$$

where:

- p : non-seasonal autoregressive (AR) order,
- d : non-seasonal differencing order,
- q : non-seasonal moving average (MA) order,
- P : seasonal AR order,
- D : seasonal differencing order,
- Q : seasonal MA order,

- s : seasonal period (e.g., $s = 7$ for weekly seasonality in daily data).

Let y_t be the differenced series after applying non-seasonal and seasonal differencing:

$$y_t = \Delta^d \Delta_s^D x_t,$$

where Δ is the differencing operator:

$$\Delta x_t = x_t - x_{t-1}, \quad \Delta_s x_t = x_t - x_{t-s}.$$

The SARIMA model can be written as:

$$\Phi_P(L^s)\phi_p(L)y_t = \Theta_Q(L^s)\theta_q(L)\varepsilon_t,$$

where:

- L is the lag operator, $Lx_t = x_{t-1}$,
- $\phi_p(L)$ is the non-seasonal AR polynomial,
- $\theta_q(L)$ is the non-seasonal MA polynomial,
- $\Phi_P(L^s)$ is the seasonal AR polynomial,
- $\Theta_Q(L^s)$ is the seasonal MA polynomial,
- ε_t is a white-noise error term.

In Python, SARIMA models are typically implemented via libraries such as `statsmodels`, where one specifies the orders (p, d, q) and (P, D, Q, s) and fits the model to the data $\{x_t\}$. The fitted model then provides forecasts $\hat{x}_{N+m}$ for $m = 1, \dots, K$.

4. Data-Analytic Algorithms and Model Selection

The practical use of SARIMA and related models requires careful **model selection** and **diagnostics**. Common steps include:

1. Stationarity Testing

Use tests such as the Augmented Dickey–Fuller (ADF) test to check whether the series is stationary. If not, apply differencing.

2. Identification of Orders

Inspect the autocorrelation function (ACF) and partial autocorrelation function (PACF) to suggest values for p, q, P , and Q .

3. Parameter Estimation

Estimate model parameters via maximum likelihood or least squares.

4. Model Diagnostics

Analyze residuals $\hat{\varepsilon}_t$ to ensure they behave like white noise:

$$\mathbb{E}[\hat{\varepsilon}_t] \approx 0, \quad \text{Cov}(\hat{\varepsilon}_t, \hat{\varepsilon}_{t-k}) \approx 0 \text{ for } k \neq 0.$$

5. Forecasting and Validation

Use out-of-sample data to validate forecasts and compute error metrics (MSE, MAE, etc.).

In a scientific setting, one often compares multiple models:

- SARIMA,
- exponential smoothing,
- state-space models,
- machine learning models (e.g., LSTM networks),

and selects the model that best balances **predictive accuracy**, **interpretability**, and **computational cost**.

5. Bayesian Reasoning and Probabilistic Forecasts

Beyond point forecasts $\hat{x}_{N+m}$, one may be interested in **probabilistic forecasts**, i.e., the distribution of future values:

$$p(x_{N+m} \mid \mathcal{D}_N).$$

Bayesian methods provide a natural framework for this. In a Bayesian time series model, we specify:

- a **likelihood** $p(\{x_t\} \mid \theta)$,
- a **prior** $p(\theta)$ over parameters θ ,
- and obtain the **posterior**:

$$p(\theta \mid \mathcal{D}_N) \propto p(\mathcal{D}_N \mid \theta)p(\theta).$$

Forecasts are then obtained by integrating over the posterior:

$$p(x_{N+m} \mid \mathcal{D}_N) = \int p(x_{N+m} \mid \theta, \mathcal{D}_N) p(\theta \mid \mathcal{D}_N) d\theta.$$

This yields not only point estimates (e.g., posterior mean) but also credible intervals, which are crucial for risk management and decision-making in finance.

6. Econometric and Physical Perspectives

Time series forecasting of stock prices is not purely a statistical exercise; it is deeply connected to **econometric theory** and, in some approaches, to **physical analogies**.

6.1 Econometric View

In econometrics, stock prices and returns are often modeled as stochastic processes such as:

- **Random walks:**

$$x_t = x_{t-1} + \varepsilon_t,$$

where ε_t is white noise.

- **AR(1) processes:**

$$x_t = \phi x_{t-1} + \varepsilon_t,$$

with $|\phi| < 1$ for stationarity.

- **GARCH models** for volatility:

$$r_t = \sigma_t z_t, \quad z_t \sim \mathcal{N}(0, 1),$$

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2.$$

These models capture different aspects of financial time series: mean dynamics, volatility clustering, leverage effects, etc.

6.2 Physical Analogies

Some approaches draw analogies between stock price dynamics and physical systems:

- **Brownian motion:**

$$S_t = S_0 \exp(\mu t + \sigma W_t),$$

where W_t is a Wiener process.

- **Langevin equations** for stochastic dynamics:

$$\frac{dX_t}{dt} = -\gamma X_t + \eta_t,$$

with η_t a noise term.

While these continuous-time models are not directly discrete time series models, they motivate discrete approximations and provide intuition about randomness, drift, and diffusion in financial markets.

7. Approximation Theory and Forecasting

Approximation theory plays a role in time series forecasting when one seeks to approximate complex dynamics with simpler models. For example, one may approximate a nonlinear function f governing the evolution:

$$x_t = f(x_{t-1}, x_{t-2}, \dots) + \varepsilon_t,$$

by a linear or polynomial model:

$$x_t \approx \sum_{k=1}^p \phi_k x_{t-k} + \varepsilon_t,$$

or by basis expansions:

$$x_t \approx \sum_{j=1}^M \beta_j \varphi_j(t),$$

where $\{\varphi_j\}$ are basis functions (e.g., Fourier basis, wavelets). Such approximations allow one to capture trends and periodicities while keeping the model tractable.

8. Stochastic Processes and Long-Term Behavior

Time series are realizations of underlying **stochastic processes**. Understanding the properties of these processes is essential for robust forecasting.

Key concepts include:

- **Stationarity:**

A process $\{X_t\}$ is (weakly) stationary if:

$$\mathbb{E}[X_t] = \mu \quad \text{constant,}$$

$$\text{Cov}(X_t, X_{t+k}) = \gamma(k) \quad \text{depends only on } k.$$

- **Ergodicity:**

Time averages converge to ensemble averages, enabling estimation from a single realization.

- **Markov property:**

Future depends only on the present, not on the full past:

$$p(X_{t+1} \mid X_t, X_{t-1}, \dots) = p(X_{t+1} \mid X_t).$$

These properties influence the choice of models and the reliability of forecasts. For example, many ARIMA models assume stationarity (after differencing), while random walk models assume non-stationarity but with specific structure.

9. Practical Implementation in Python

In practice, forecasting stock prices or other time series using SARIMA and related models is implemented in Python using libraries such as:

- **pandas** for data handling,
- **numpy** for numerical operations,
- **statsmodels** for SARIMA and ARIMA,
- **scikit-learn** for machine learning models.

A typical workflow:

1. **Data Preparation**

Load $\{x_t\}$, handle missing values, transform to returns if needed.

2. **Exploratory Analysis**

Plot the series, compute ACF/PACF, test for stationarity.

3. **Model Specification**

Choose SARIMA orders (p, d, q) and (P, D, Q, s) .

4. **Model Fitting**

Fit the model to $\{x_t\}_{t=1}^N$.

5. **Forecasting**

Generate forecasts $\hat{x}_{N+m}$ for $m = 1, \dots, K$.

6. Evaluation

Compare forecasts to actual values, compute error metrics, refine the model.

10. Reliability, Limitations, and Future Directions

While SARIMA and related models provide a powerful framework for time series forecasting, especially for stock prices and financial indices, they have limitations:

- They assume that past patterns will persist into the future.
- They may struggle with structural breaks, regime changes, or extreme events.
- They often require stationarity, which may not hold in raw price series.

To improve predictive reliability, one may:

- Combine SARIMA with **machine learning models** (hybrid approaches).
- Incorporate **exogenous variables** (e.g., macroeconomic indicators, sentiment scores).
- Use **state-space models** and **Kalman filters** for dynamic parameter estimation.
- Explore **Bayesian time series models** for probabilistic forecasts and uncertainty quantification.

In all cases, the fundamental question remains:

Given

$$\{x_t\}_{t=1}^N,$$

how can we best estimate

$$\{x_{N+m}\}_{m=1}^K$$

under uncertainty?

The answer lies in a careful combination of **statistical theory**, **computational methods**, and **domain knowledge**, supported by rigorous validation and transparent reporting.
